Project NordForge — End-to-End Corporate Cybersecurity Program

Final Project — 420-4C2-DW, Implementation & Management of Corporate Cybersecurity Measures

Cyber.SoHo Educational Hub — "From the Industry to the Classroom — Building the IT's Future, Today and Together!"

Instructor: Asen Grozdanov **Cohort:** Cybersecurity: Prevention and Intervention — LEA.D8 **Delivery format:** Pairs, on VirtualBox, with pre-recorded PowerPoint presentation **Window:** 30 May – 26 June **Weight:** 70 % of the course grade (Stages + Final Project + Presentation) + 30 % Final Exam

---

0. Read this before anything else

Hello, and welcome to what is — and I don't say this lightly — the most important single piece of work you will produce in this program. The capstone course, 420-4C2-DW, is built to force you to stitch together **seven** prior courses into one coherent cybersecurity program for a realistic company. Not seven little projects stapled together. **One program**. A small SOC (Security Operations Center) will tell you the same thing: the hardest part of this job isn't learning a tool — it's making the tools, the policies, the people, and the paperwork all pull in the same direction when something goes wrong at 2:37 a.m. on a Tuesday.

A few ground rules before the fun starts.

- You work in **pairs**. I pick the pairs. No negotiation. You'll be forced to communicate, divide labour, and — most importantly — cover for each other when one of you gets stuck.
- All work happens inside **Oracle VirtualBox** on your own machine. No cloud spillage, no production systems, no toys pointed at anyone's real network. Ever.
- You will deliver **four Stage submissions**, a **Final Document** with full screenshots and commentary, and a **PowerPoint presentation** where **each partner records their own part of the narration** using PowerPoint's built-in recording tool. I want to hear your voice explaining your slides, in your own words — this is non-negotiable.
- Every abbreviation you use, first time you use it, must be written in full **beside** the abbreviation — for example: *IR (Incident Response)*, *IOC (Indicator Of Compromise)*,

CVSS (Common Vulnerability Scoring System). You will see I do the same in this brief. Get used to it.

- Every organisation, standard, tool, or document you reference must be **hyperlinked** to its official site. If you put "PCI-DSS" in your paper without a link to the PCI Security Standards Council, you lose a mark. Every time.
- You are **free to substitute tools, operating systems, or scripts** with alternatives you prefer — *provided* you justify **why** in writing. If you built something yourself, you submit the **source code**, clearly commented. You may also replace the infrastructure I give you with a **more complex one**, again provided you can demonstrate in writing that it really is more complex, not merely louder.

One more thing. Cybersecurity is a discipline where copying someone else's work, letting a language model write your report, or running tools you don't understand will get you kicked out of the industry before your first paycheque clears. I have been doing this for about twenty-five years, and I can spot a student who cut corners inside of five minutes of their presentation. **Do the work. Learn the tools. Explain yourself.**

---

1. The Brief — what you are being hired to do

You and your partner are a two-person cybersecurity consultancy. A company has retained you for a fixed-price engagement to deliver a complete cybersecurity program: risk analysis, defensive architecture, a penetration test on your own build, an incident response capability, and a user-awareness push. At the end of the engagement you present your work to the board (me, and your classmates).

The four stages map cleanly onto the four competency elements of EZ08 for this course:

Stage	Theme	Deliverable deadline (Session)	Course competency element
Stage 1	*Know thy network* — risk analysis, assets, threats, baseline monitoring	Thu 04 Jun, 18:00–22:00	**E1.** Develop a cybersecurity incident response plan
Stage 2	*Harden and secure* — defence-in-depth, vulnerability remediation, compliance	Thu 11 Jun, 18:00–22:00	**E2.** Secure information
Stage 3	*Test the defence* — authorised penetration test against your own build	Thu 18 Jun, 18:00–22:00	**E1/E2 validation.** Prove your controls work
Stage 4	*Detect, respond, recover & educate* — incident management + user awareness	Tue 23 Jun, 18:00–22:00	**E3.** Manage incidents + **E4.** Raise user awareness

That last column is why this project exists at all. The EZ08 competency requires all four elements in one integrated exercise. Doing them in isolation doesn't count.

1.1 Full sixteen-session schedule

#	Date	Hours	Session focus
S1	Sat **30 May**	09:00 – 12:00	**Kickoff:** brief, pair assignments, VirtualBox sanity check, obtain installers
S2	Sat **30 May**	13:00 – 17:00	Lab build — OPNsense (Open-source Network Security) firewall + VLANs (Virtual Local Area Networks)
S3	Sun **31 May***	09:00 – 12:00	Lab build — remaining VMs (Virtual Machines); asset inventory begins
S4	Mon **01 Jun**	18:00 – 22:00	Threat modeling, risk register, baseline Wireshark + Nmap (Network Mapper)
S5	**Thu 04 Jun**	**18:00 – 22:00**	**STAGE 1 DUE — upload + instructor walk-through**
S6	Fri **05 Jun**	18:00 – 22:00	Firewall rules, VLAN segmentation, Fail2Ban, SSH (Secure Shell) hardening
S7	Sat **06 Jun**	13:30 – 17:30	Windows + AD (Active Directory) hardening, LAPS (Local Administrator Password Solution), GPOs (Group Policy Objects)
S8	Mon **08 Jun**	18:00 – 22:00	Vulnerability scan with OpenVAS, CVSS triage, OpenSCAP + Lynis compliance scan
S9	**Thu 11 Jun**	**18:00 – 22:00**	**STAGE 2 DUE — upload + instructor walk-through**
S10	Sat **13 Jun**	14:30 – 18:30	Pentest — RoE (Rules of Engagement), OPSEC (Operational Security), passive + active recon
S11	Mon **15 Jun**	18:00 – 22:00	Pentest — exploitation on Metasploitable + DVWA (Damn Vulnerable Web Application), AD attacks
S12	**Thu 18 Jun**	**18:00 – 22:00**	**STAGE 3 DUE — upload + instructor walk-through**
S13	Fri **19 Jun**	18:00 – 22:00	Incident Response plan, playbooks, Wazuh alert tuning from pentest artefacts
S14	Sat **20 Jun**	12:00 – 17:00	Full IH&R (Incident Handling & Response) simulation; forensic evidence; awareness material
S15	**Tue 23 Jun**	**18:00 – 22:00**	**STAGE 4 DUE — Final document + pre-recorded presentation submitted**
S16	**Fri 26 Jun**	**18:00 – 22:00**	**FINAL WRITTEN EXAM — covers all seven integrated courses**

*The schedule provided to me listed a second "Saturday 30 May, 09:00–12:00" which I have read as **Sunday 31 May, 09:00–12:00** (S3) — two three-hour sessions on a single calendar day would overlap. If the intent was a different date, we adjust it on day one.*

1.2 Evaluation breakdown

Component	Weight
Stage 1 submission (Thu 04 Jun)	7 %
Stage 2 submission (Thu 11 Jun)	7 %
Stage 3 submission (Thu 18 Jun)	7 %
Stage 4 submission (Tue 23 Jun)	7 %
Subtotal — staged submissions	**28 %**
Final Project document (Tue 23 Jun)	30 %
Pre-recorded PowerPoint presentation (Tue 23 Jun)	12 %
Subtotal — final deliverable	**42 %**
Final written exam (Fri 26 Jun)	30 %
Total	**100 %**

Minimum passing grade is 60 %. You also have to pass the summative element on its own. If you crush the stages but flame out on the final presentation, you fail the course. That's the rule, and I enforce it.

---

2. The fictitious client — NordForge Industries

*Every detail in this section is fabricated. Any resemblance to a real company is coincidence. See the **Disclaimer** at the end.*

NordForge Industries Inc. is a mid-sized Quebec-based precision-forging manufacturer with roughly 180 employees across three sites: a head office in Montréal, a production plant in Trois-Rivières, and a small R&D (Research & Development) office in Boston. They sell specialty forged components to the aerospace and heavy-equipment sectors.

Their IT footprint — the one *you* will model in the lab — is modest but realistic:

- An **on-prem (on-premises) Active Directory** domain, `corp.nordforge.local`, with about 200 user accounts and a dozen servers.
- A **perimeter firewall** protecting the corporate network from the internet, with a small **DMZ (Demilitarized Zone)** hosting a customer-facing order portal and a public website.
- A small **SOC / monitoring segment** that you'll spin up on a separate VLAN.

- A mix of **Windows 10/11 workstations**, a couple of **Ubuntu Server 22.04** boxes for internal web apps and file sharing, and at least one **deliberately vulnerable host** so you have something to attack in Stage 3.
- A **remote-worker VPN (Virtual Private Network)** — optional for students who want to raise the complexity bar.

Two weeks before the engagement starts, NordForge had an embarrassing incident: an accountant's laptop received a phishing email impersonating a supplier, and the attacker briefly gained access to a shared folder. IT killed the session quickly, but the CEO is now spooked. They want:

1. A clear picture of what they own, what's at risk, and how risky.
2. Controls that prove the next attempt gets blocked or detected.
3. A plan for what to do when — not if — the *next* intrusion happens.
4. Their staff not clicking on phishing again.

That's the engagement. Now build it.

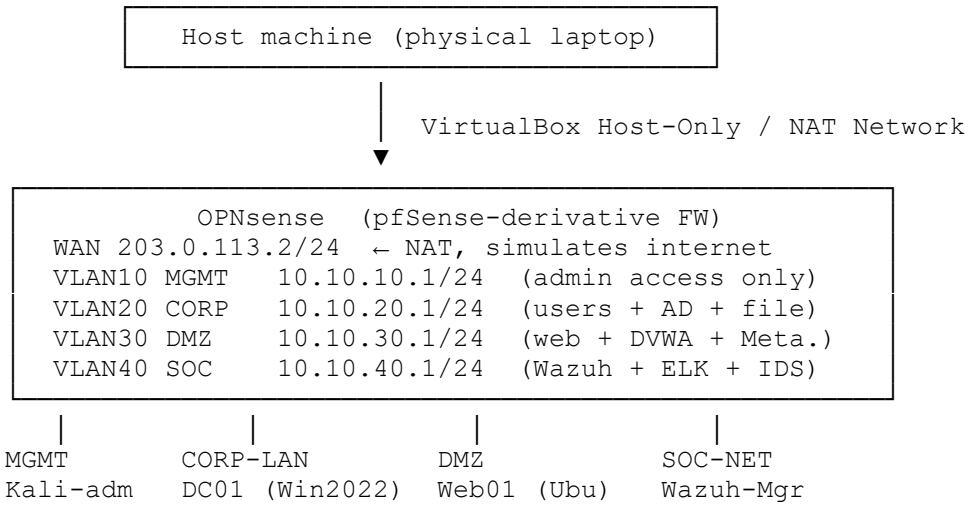
---

3. The lab infrastructure (VirtualBox)

The whole lab lives on a single physical host with at least **16 GB RAM** and **200 GB free disk**. If you don't have that, pair up on one machine and agree a revision-control routine early.

3.1 Network design

Five VLANs behind a single OPNsense firewall. The attacker network is a **separate** NAT network so that Stage 3 traffic flows through the firewall's WAN (Wide Area Network) interface the same way a real external attacker would — not magically through the LAN (Local Area Network) back door.



```
                WKS01 (Win10)    DVWA-Meta      Kibana-ELK
                FS01 (Ubuntu)                   Suricata-sensor

   ATTACKER (separate NAT network 198.51.100.0/24)
      Kali-ext     ← This is the "internet attacker" in Stage 3
      Whonix GW    ← Optional OPSEC layer
```

That ASCII sketch is the simplest honest drawing of the topology. In your submission, please re-draw it in draw.io / diagrams.net or Excalidraw with labels on each interface. A prettier picture has not saved a single engagement from disaster — but a messy one has caused plenty. Clarity matters.

3.2 Address plan (RFC 1918 — private addressing)

A tiny spreadsheet you will expand in Stage 1. This lives as `spreadsheets/asset_inventory_template.csv` in your submission bundle.

VLAN	Purpose	Subnet	Gateway	DHCP range
10	MGMT — firewall + switch admin	10.10.10.0/24	10.10.10.1	static only
20	CORP — users, AD, file server	10.10.20.0/24	10.10.20.1	.100 – .200
30	DMZ — public-facing services	10.10.30.0/24	10.10.30.1	static only
40	SOC — Wazuh, ELK, IDS sensor	10.10.40.0/24	10.10.40.1	static only
—	WAN — simulated ISP uplink	203.0.113.0/24	203.0.113.1	—
—	ATTACKER net — external Kali	198.51.100.0/24	198.51.100.1	—

Using TEST-NET-1, TEST-NET-2, TEST-NET-3 (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`) for your "public" side is an old hygiene habit. It makes sure that if, by accident, something ever escapes your lab, it routes precisely nowhere. Do it.

3.3 Virtual machine list — recommended builds

The "recommended" column is what I run in class. The "why this one" column explains the pedagogical choice. If you swap it for something more ambitious or more obscure (say, Qubes OS for the attacker instead of Kali), you **must** explain why you prefer it — one paragraph, in Stage 1.

Role	Recommended VM	Why this one	Min. resources
Perimeter FW (Firewall)	OPNsense 24.7	Open source, VLAN-aware, Suricata and Zenarmor plugins, cleaner UI than pfSense	2 vCPU, 2 GB, 20 GB
Domain Controller	Windows Server 2022 Eval (90-day eval)	Real AD, real GPO, matches what you'll see at any SME	2 vCPU, 4 GB, 40 GB

Role	Recommended VM	Why this one	Min. resources
Windows Workstation	Windows 10 Enterprise Eval (eval link)	Domain-joined target for phishing and privilege escalation	2 vCPU, 4 GB, 40 GB
Linux File Server	Ubuntu Server 22.04 LTS	Industry-default Linux, excellent Wazuh agent support	1 vCPU, 2 GB, 20 GB
DMZ Web Server	Ubuntu Server 22.04 + DVWA + OWASP Juice Shop	Purpose-built vulnerable apps for OWASP Top 10 practice	2 vCPU, 2 GB, 20 GB
Deliberately vuln host	Metasploitable 2	Classic CTF (Capture The Flag) target with known exploits	1 vCPU, 512 MB, 8 GB
SIEM (Security Information & Event Management)	Wazuh 4.x all-in-one	Covers HIDS (Host-based IDS), FIM (File Integrity Monitoring), and SIEM in one	4 vCPU, 8 GB, 60 GB
Network IDS (Intrusion Detection System) sensor	Security Onion 2 *or* Suricata on OPNsense	Signature + anomaly detection, full PCAP (Packet Capture)	2 vCPU, 4 GB, 40 GB
Attacker — internal admin	Kali Linux	Standard pentest distro, Metasploit + nmap + Burp	2 vCPU, 4 GB, 40 GB
Attacker — external	Kali Linux or Parrot OS + Whonix Gateway	Simulates an internet-based threat actor, with optional OPSEC	2 vCPU, 4 GB, 40 GB

VirtualBox version note: the project is tested on VirtualBox 7.0.x. If you use VMware Workstation Pro (now free for personal use since May 2024) or KVM/QEMU, that's fine — justify the choice and show us the VM export format you'll hand in.

3.4 Save snapshots early and often

Nothing breaks a student's weekend like losing six hours of OPNsense config to a botched firewall rule. Take a **VirtualBox snapshot** (VirtualBox menu → *Machine* → *Take Snapshot*) before every Stage submission, and before every experimental change. From the command line that's:

```
# ------------------------------------------------------------------
# Take a named snapshot of a running VM. Run this from the host shell.
# ------------------------------------------------------------------
VBoxManage snapshot "OPNsense"  take  "Stage1-clean"  \
                                      --description "Baseline before firewall
rule work"
# Replace "OPNsense" with the VM name as shown by:   VBoxManage list vms
```

That one-liner will save you from yourself at least three times during the project. I guarantee it.

---

4. Stage 1 — *Know thy network* (due Thu 04 Jun, Session 5)

Weight: 7 % **Covered courses:** 420-1C1-DW Preventative Monitoring · 420-1C2-DW Threats and Risks · parts of 420-3C1-DW (asset discovery) **Competency element:** EZ08-1 *Develop a cybersecurity incident response plan — identification of relevant sources, methodical collection and analysis, prioritization of assets, identification of main threats and vulnerabilities.*

You can't defend what you don't know you have. That sentence is a cliché in this industry precisely because people keep forgetting it. Stage 1 forces you to produce the evidence that you actually know your network.

4.1 Objectives of Stage 1

1. Stand up the full VirtualBox lab from section 3.
2. Build an **asset inventory** — every VM, every service, every account of note.
3. Run **baseline network reconnaissance** from inside the lab — Nmap sweep, Wireshark capture, a short walk-through of what "normal" looks like.
4. Produce a **threat model** using STRIDE and/or OWASP Threat Dragon.
5. Produce a **risk register** using NIST SP 800-30 Rev. 1 likelihood × impact, cross-referenced with ISO 31000:2018 terminology.
6. Stand up **preventative monitoring**: Wazuh agents on every host, Suricata on OPNsense, a Kibana dashboard showing the baseline.

4.2 Stage 1 — step-by-step

Step 1.1 — Install OPNsense and carve up the VLANs

OPNsense is a FreeBSD-based open-source firewall forked from pfSense in 2014. Install it with two network interfaces in VirtualBox: `vtnet0` bound to a NAT network named `wan-sim`, and `vtnet1` bound to an Internal Network named `corp-trunk`. In OPNsense's initial console you assign them to **WAN** and **LAN**. Once you log into the web UI at `https://10.10.20.1` (reachable from a host-only adapter during setup), you create the VLAN tags on `vtnet1`:

- Go to **Interfaces → Other Types → VLAN**.
- Add tags **10 / 20 / 30 / 40** on parent interface `vtnet1`.
- Assign each new VLAN interface and give them the addresses from the table in §3.2.

Why VLANs at all? Because without them, a phished workstation on the user network and the domain controller share a broadcast domain, and an attacker who lands on any Windows box gets to say hello to everything at L2 (Layer 2). We'll prove that point in Stage 3. For now, just give yourself the option to segregate.

A quick ASCII reminder of what VLAN segmentation looks like in traffic terms:

```
          Without VLAN segmentation           With VLAN segmentation
          -------------------------           ----------------------
                (shared LAN)                        (per-VLAN broadcast)
       Attacker -----> DC01                Attacker --X--> Firewall rule
drops
       Attacker -----> WKS01                Attacker -----> DC01 only if ACL
allows
       Attacker -----> File server          East-west traffic requires
explicit rule
```

That "`--X-->`" is the single biggest architectural improvement you can make in an hour of work.

Step 1.2 — Boot the rest of the fleet

Install the remaining VMs from §3.3. Drop them onto the correct VLAN by editing each VM's network adapter:

- **Adapter 1 → Internal Network** → pick `vlan20-corp` or `vlan30-dmz` etc.
- Make sure **Promiscuous Mode** is set to **Allow VMs** on the SOC VLAN (you'll need it for Suricata sensor traffic).

Once Windows Server 2022 boots, promote it to a domain controller:

```
# ----------------------------------------------------------------------------
--
# Promote the server to the first Domain Controller in a new forest.
# Run from an elevated PowerShell on the server console.
# ----------------------------------------------------------------------------
--
Install-WindowsFeature  -Name AD-Domain-Services  -IncludeManagementTools
# Install the AD DS role + RSAT tools
Import-Module           ADDSDeployment
# Bring the deployment cmdlets into scope
Install-ADDSForest  `
    -DomainName                     "corp.nordforge.local"  `
# The fake corporate domain name
    -DomainNetbiosName              "CORP"                  `
# Legacy NetBIOS name — keep it short
    -ForestMode                     "WinThreshold"          `
# Current forest functional level
    -DomainMode                     "WinThreshold"          `
# Same for domain
    -InstallDns                     $true                   `
# AD DS really needs its own DNS
    -DatabasePath                   "C:\Windows\NTDS"       `
    -LogPath                        "C:\Windows\NTDS"       `
    -SysvolPath                     "C:\Windows\SYSVOL"     `
    -Force                          $true
# Don't prompt — we're automating
# Server will reboot automatically. After reboot, join WKS01 to the domain.
```

I comment every single line because in production I review my own scripts three months later and cannot remember why I did anything. You will be in the same boat. Get the habit now.

Step 1.3 — Build the asset inventory

Open `spreadsheets/asset_inventory_template.csv` (in your project bundle). Fill in a row per host. At minimum you track:

- Hostname, IP, OS (Operating System) + version, owner, purpose
- Services / listening ports (use `ss -tlnp` on Linux, `Get-NetTCPConnection -State Listen` on Windows)
- Data classification: **Public / Internal / Confidential / Restricted**
- Criticality (1–5) using ISO 27005:2022 language: impact if the asset loses **C**onfidentiality, **I**ntegrity, **A**vailability — the **CIA Triad**

Real-world asset managers like GLPI or Snipe-IT shine once you have a couple hundred assets, but for this project a careful CSV is more than enough.

Step 1.4 — Baseline reconnaissance (from inside)

Fire up Kali-adm on VLAN10 and run a friendly, *internal* sweep. This is **not** a penetration test. It's an inventory check with a sharper tool than `arp -a`:

```
#!/usr/bin/env bash
# ------------------------------------------------------------------------
--
# Lab baseline discovery -- run from Kali-adm, NOT from the attacker network.
# Output goes to ~/nordforge/stage1/ for inclusion in your Stage 1 PDF.
# ------------------------------------------------------------------------
--
set -euo pipefail                                   # Fail loudly on errors
OUT=~/nordforge/stage1
mkdir -p "$OUT"

# 1) ARP table of every VLAN we reach (quick & quiet)
for v in 10 20 30 40 ; do
  sudo nmap -sn -PR  10.10."$v".0/24  -oN  "$OUT/ping-vlan${v}.nmap"
done

# 2) Full TCP port sweep on discovered hosts, versions + default scripts
sudo nmap  -sS  -sV  -sC  --top-ports 1000  --open  \
           -iL <(awk '/Nmap scan report/ {print $5}' "$OUT"/ping-vlan*.nmap)
\
           -oA  "$OUT/svc-discovery"

# 3) One lightweight UDP sweep -- UDP discovery is slow, so only top 50 ports
sudo nmap  -sU  --top-ports 50  -iL <(awk '/Nmap scan report/ {print $5}'
"$OUT"/ping-vlan*.nmap) \
           -oA  "$OUT/udp-top50"
echo  "Baseline complete. Review the -oA files before Stage 1 submission."
```

A few intentional choices in that script:

- `-sS` (SYN scan) is faster and lighter than `-sT` and is the default for privileged users. Do not use it on networks you don't own.
- `-sC` runs the default NSE (Nmap Scripting Engine) scripts. That's enough to enumerate SMB (Server Message Block) versions, SSL (Secure Sockets Layer) certificates, and SSH banners without being noisy.
- UDP is slow. Don't try to sweep 65 535 UDP ports. You'll age.

Capture five minutes of "normal" traffic on VLAN20 with Wireshark (or `tshark -i vlanXX -w baseline.pcap`) and keep that PCAP. In Stage 3 you'll compare it to pentest traffic, and the contrast is the whole point.

Step 1.5 — Threat modeling with STRIDE

STRIDE, created by Loren Kohnfelder and Praerit Garg at Microsoft in 1999, is still the simplest way to brainstorm attack categories. Build a data-flow diagram of the NordForge network in OWASP Threat Dragon — it's free, open source, runs as a desktop app, and exports to JSON.

The six STRIDE categories and what they map to on your network:

Threat	Meaning	NordForge example
Spoofing	Pretending to be another identity	Attacker forges an SMB (Server Message Block) authentication to FS01
Tampering	Unauthorised modification	Attacker alters a payroll CSV on the file server
Repudiation	Denying an action	User deletes evidence; no log retention
Information disclosure	Leak of confidential data	DVWA leaks database contents via SQLi
Denial of service	Availability attack	SYN flood against the DMZ web server
Elevation of privilege	Gaining unauthorised rights	Kerberoasting against a service account with a weak password

For each data flow on your diagram (user → web server, web server → database, etc.) you note which STRIDE threats apply and how you might mitigate them. This is a **one-hour exercise** if you're disciplined. Don't agonise over "perfection"; agonise over completeness.

Step 1.6 — Risk register

A risk register is a living document. Yours starts at Stage 1 and grows until Stage 4. Format it per `spreadsheets/risk_register_template.csv`. Each row contains:

- Risk ID
- Description
- Threat source (internal / external / natural / supply-chain)
- Likelihood (1–5) and rationale

- Impact (1–5) and rationale
- Inherent risk score (L × I)
- Current controls
- Residual risk score
- Treatment decision: **Avoid / Reduce / Transfer / Accept** (NIST 800-39 and ISO 31000 terminology)
- Owner, target date, status

Use the 5×5 matrix below — it's the same one the FAIR Institute uses in its basic materials and what CRA (Canada Revenue Agency) public documents show for operational risk.

```
                          IMPACT →
                   1(Low) 2      3      4      5(Critical)
LIKELIHOOD ↓  1(Rare)   1      2      3      4      5
              2         2      4      6      8     10
              3         3      6      9     12     15
              4         4      8     12     16     20
              5(Almost  5     10     15     20     25
                 certain)

                  1-4   = Low (green)     — accept or monitor
                  5-9   = Medium (yellow) — reduce within quarter
                 10-14  = High (amber)    — reduce this month
                 15-25  = Critical (red)  — immediate action
```

Step 1.7 — Preventative monitoring: Wazuh, Suricata, Kibana

Install Wazuh Manager as an all-in-one on the SOC VLAN40 (follow the official all-in-one installer). Then install Wazuh agents on every Linux and Windows host. The agent install on Ubuntu is a two-liner:

```
# ------------------------------------------------------------------------
--
# Register a Linux host to Wazuh Manager on 10.10.40.10. Replace the IP if
needed.
# Run as root on FS01, Web01, etc.
# ------------------------------------------------------------------------
--
curl -s  https://packages.wazuh.com/key/GPG-KEY-WAZUH  \
  | gpg --no-default-keyring --keyring gnupg-
ring:/usr/share/keyrings/wazuh.gpg --import
chmod 644 /usr/share/keyrings/wazuh.gpg
# Must be world-readable for apt
echo  "deb [signed-by=/usr/share/keyrings/wazuh.gpg]
https://packages.wazuh.com/4.x/apt/ stable main" \
  >  /etc/apt/sources.list.d/wazuh.list

WAZUH_MANAGER="10.10.40.10"  apt-get update  &&  apt-get install -y  wazuh-
agent      # Agent picks up the env var
systemctl enable  --now  wazuh-agent
```

Turn on Suricata inside OPNsense (**Services → Intrusion Detection**) and enable the Emerging Threats Open ruleset. You'll want alerts — not blocks — in Stage 1; you are *listening*, not *intervening*, until Stage 2.

Wazuh ships a Kibana dashboard out of the box. Take a screenshot of it showing the first 24 hours of baseline events — logons, failed authentications, firewall drops. That screenshot is your Stage 1 "proof that monitoring exists" artefact.

Step 1.8 — Video reference (optional but recommended)

If you want a second pair of eyes on the threat-modelling material, OWASP's own Threat Dragon demo playlist on YouTube is solid. On VirtualBox-specific VLAN setup, Lawrence Systems' channel has clear walkthroughs. Their licensing is standard YouTube fair-use — include a short citation in your doc, do not re-host the video.

4.3 Stage 1 deliverables — what to hand in on 04 Jun

A **single ZIP file** named `TeamXX_Stage1.zip` containing:

1. `Stage1_Document.pdf` — your narrative (est. 15–25 pages) including:
    - Client intro + your interpretation of the brief
    - Architecture diagram (re-drawn, clean)
    - Filled-in asset inventory CSV + rationale
    - Filled-in risk register CSV (minimum 15 risks)
    - STRIDE threat model (exported from Threat Dragon)
    - Nmap + Wireshark baseline screenshots
    - Screenshot of Wazuh dashboard with at least 24 h of data
    - Short reflection: what surprised you; what you had to substitute and why
2. `assets/asset_inventory.csv` — the living file
3. `assets/risk_register.csv` — the living file
4. `evidence/baseline.pcap` + `evidence/nmap/*.nmap` — raw outputs
5. Any custom scripts you wrote with full comments
6. `readme.md` — short guide to your ZIP's contents

Grading hotspots (what separates a 70 % Stage 1 from a 95 % one): the *depth* of the risk register, the *honesty* of the reflection, and whether the threat model actually matches the diagram.

---

5. Stage 2 — *Harden and secure* (due Thu 11 Jun, Session 9)

Weight: 7 % **Covered courses:** 420-2C1-DW Cyber Defence · 420-3C1-DW Managing Vulnerabilities · Compliance Assessments **Competency element:** EZ08-2 *Secure information — correct application of norms/policies/standards, correction of high-risk vulnerabilities, access control, validation of effectiveness.*

Stage 1 produced a map. Stage 2 is where you pick up the tools and actually fix things. You also write down — in plain language — **why** you did what you did, so a future auditor does not have to guess.

5.1 Objectives of Stage 2

1. Apply **perimeter defence** — firewall rules, inter-VLAN segmentation.
2. **Harden** every host — Linux *and* Windows — to a published baseline.
3. Harden **Active Directory**: the most abused part of any corporate network.
4. Run a **vulnerability scan**, triage findings with CVSS, and produce a **patch plan** with evidence of remediation.
5. Run a **compliance scan** against CIS Benchmarks and DISA STIG profiles using OpenSCAP and Lynis.
6. Map your residual gaps to two regulatory frameworks relevant in Canada: **PIPEDA** and **Bill C-26 / Critical Cyber Systems Protection Act**, and one international: **NIST SP 800-53 Rev. 5**.
7. Draft three **policies**: Access Control, Acceptable Use, and a first-draft Incident Response Plan (you will refine the IRP in Stage 4).

5.2 Stage 2 — step-by-step

Step 2.1 — Perimeter defence on OPNsense

Default posture is **deny all, allow explicitly**. In the OPNsense rule editor (*Firewall → Rules*) this is the tail rule on every interface:

```
Action:  Block
Interface: LAN / VLAN20 / VLAN30 / VLAN40 / MGMT   (each one)
Protocol: any
Source:   any
Destination: any
Log:      yes
Description: "Default deny -- everything not explicitly permitted"
```

Then you *add back* only the flows the business needs. An abbreviated matrix, which you'll expand in your submission:

Source VLAN	Destination	Ports	Reason
VLAN20 CORP	VLAN10 MGMT	**none**	Users should never touch the management plane
VLAN20 CORP	DC01	53, 88, 135, 389, 445, 464, 636, 3268-3269, 49152-65535	Standard AD ports
VLAN20 CORP	FS01	445 (SMB), 22 (mgmt from admins only)	File share
VLAN20 CORP	Internet (WAN)	80, 443, 53	Web browsing + DNS

Source VLAN	Destination	Ports	Reason
VLAN30 DMZ	VLAN20 CORP	**none**	DMZ must never initiate into CORP
VLAN30 DMZ	Internet (WAN)	80, 443, 53	Updates only
Internet → VLAN30 DMZ	80, 443	DNAT (Destination NAT)	Public web + portal
VLAN40 SOC	all VLANs	chosen monitoring ports	SIEM collection only
Any	OPNsense itself	none (except MGMT → web UI)	Management plane lockdown

In the MGMT interface rules you **restrict web-UI access** to the two admin IPs only. In production the same idea applies — the firewall's own admin port is the single juiciest target on the network.

Step 2.2 — Linux hardening

Run `bash/02_linux_hardening.sh` (included in your bundle) on Ubuntu Server 22.04. It does the following things that every CIS Benchmark would have you do anyway, plus a few extras I picked up running servers for a living:

- Disable root login over SSH, change the SSH port off 22 to a four- or five-digit non-privileged port, and require key-based authentication.
- Install and configure Fail2Ban with a strict SSH jail (5 failures in 10 minutes → 24-hour ban).
- Enable UFW (Uncomplicated Firewall) as a last-line host firewall, allowing only your new SSH port + whatever the box's role requires.
- Set `PermitEmptyPasswords no`, `UseDNS no`, `ClientAliveInterval 300`, and disable legacy KEX / MAC / cipher suites per Mozilla SSH guidelines.
- Mask unused services: `cups`, `avahi-daemon`, `bluetooth`, `ModemManager` (unless you have a reason).
- Enable `auditd` with the Linux Audit rules for CIS Level 1.
- Enforce a strong `/etc/login.defs` password policy: min length 14, complexity via `libpam-pwquality`, account lockout via `pam_faillock`.

Once you run the script, take a **before / after** Lynis scan and include both reports. Lynis gives you a *hardening index* out of 100; on a fresh Ubuntu server you start around 55 and a first hardening pass typically lifts you into the low 80s. Above 90 starts requiring trade-offs; know which trade-offs you are willing to make.

Step 2.3 — Windows hardening

On DC01 and WKS01, apply a baseline via Group Policy. The simplest trustworthy source is the Microsoft Security Compliance Toolkit, which ships the *Windows 10 Security Baseline* and the

Windows Server 2022 Security Baseline as importable GPO backups. Download, extract, and run `LGPO.exe` to apply them on the workstation, or import them into Group Policy Management on DC01 for the domain.

Your Stage 2 submission must also show these specific wins, which I will ask about during the presentation:

- **SMB v1 disabled** on every Windows host (`Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol`). The WannaCry worm in May 2017 propagated over SMB v1; there is no excuse to have it enabled in 2026.
- **LM and NTLMv1 authentication disabled** via GPO (`Computer Configuration → Windows Settings → Security Settings → Local Policies → Security Options → Network security: LAN Manager authentication level = Send NTLMv2 response only. Refuse LM & NTLM`).
- **LAPS (Local Administrator Password Solution)** deployed (Windows LAPS is now built into modern Windows). This randomises the local Administrator password on every machine and stores it in AD, which makes Pass-the-Hash from WKS01 against DC01 a lot less fun for an attacker.
- **Windows Defender Application Control** or at minimum AppLocker blocking executables from user-writable locations (`%TEMP%`, `%APPDATA%`). Most commodity malware dies here.

See `powershell/01_windows_baseline.ps1` in your bundle for a scripted starting point.

Step 2.4 — Active Directory hardening

This is where pentesters earn their keep. AD is a twenty-five-year-old distributed database with backwards-compatibility warts that the Red Team side of the house has been mapping out publicly since at least 2014 (see the BloodHound project).

Minimum hardening actions for your project, all documented in `powershell/02_ad_audit.ps1`:

- Move privileged users to a **Protected Users** group. This disables NTLM for them, disables cached credentials, and forces AES-only Kerberos.
- Remove **SPN (Service Principal Name)** from regular user accounts — a user with an SPN is Kerberoastable.
- Set every **service account** password to 32+ characters, random, stored in a vault (we'll use HashiCorp Vault community edition in class, or KeePassXC if the team prefers).
- Disable **AS-REP roasting** — clear the `DONT_REQ_PREAUTH` flag on every user (`Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true}`).
- Enable **Kerberos AES encryption only** on all service accounts (`msDS-SupportedEncryptionTypes` = 0x18).
- Create a **tiered admin model**: Tier 0 admins (Domain Admin) log on only from Tier 0 hosts (DC01 itself, or a jump box). Tier 1 admins manage servers. Tier 2 admins manage

workstations. The cross-tier prohibition is what neutralises most privilege-escalation paths.
- Enable **Advanced Audit Policy** on DC01 for: Account Logon, Logon/Logoff, Directory Service Access, Kerberos Service Ticket Operations. These are the events Wazuh will correlate in Stage 4.

Install BloodHound Community Edition and run `SharpHound.exe -c All` against your hardened AD — capture the graph, identify the shortest path from your lowest-privilege user to Domain Admin, and write a paragraph on what that path is and how you mitigated it. If the shortest path is "zero edges" (you are already Domain Admin) — you've either not hardened enough, or you scanned with the wrong account. Redo it.

Step 2.5 — Vulnerability scanning and CVSS triage

Install Greenbone Community Edition / OpenVAS on the SOC VLAN. Let the feed sync (it takes 30–60 minutes the first time — grab a coffee). Run an **authenticated credentialed scan** against all hosts in VLAN20 and VLAN30:

```
# ------------------------------------------------------------------------
--
# Example: trigger an OpenVAS scan from CLI via gvm-cli (after container is
up)
# This uses the GVM (Greenbone Vulnerability Management) protocol over TLS.
# ------------------------------------------------------------------------
--
gvm-cli  \
  --gmp-username  admin  \
  --gmp-password  "$(cat  /run/secrets/gvm-admin)"  \
  tls --hostname 10.10.40.20 --port 9390  \
  --xml  "<create_target>
            <name>NordForge VLAN20</name>
            <hosts>10.10.20.0/24</hosts>
          </create_target>"
# Then create a task, run it, and export the report as CSV for triage.
```

Triage every finding into `spreadsheets/cvss_triage.csv`, with columns:

- CVE (Common Vulnerabilities and Exposures) ID, if any — linked to MITRE CVE and/or NVD (National Vulnerability Database)
- Asset, service, port
- CVSS (Common Vulnerability Scoring System) v3.1 Base score — computed from the FIRST.org calculator
- Environmental score adjustment (you, in *your* environment — this is the bit most students skip)
- Exploitability in the wild — is there a public PoC (Proof of Concept)? Check Exploit-DB and CISA's Known Exploited Vulnerabilities (KEV) list
- MITRE ATT&CK technique mapped (fill from `reference/mitre_attack_mapping.csv`)
- Remediation action: patch / config / mitigate / accept

- Target date + owner + evidence of completion

You will not patch everything. That is realistic. What matters is that you can **defend** your decisions. "We accepted this because patching requires vendor update that drops in Q3" is a professional answer. "We accepted this because we ran out of time" is not.

Step 2.6 — Compliance scanning with OpenSCAP and Lynis

OpenSCAP evaluates a host against a published SCAP (Security Content Automation Protocol) profile. On Ubuntu 22.04:

```
# ------------------------------------------------------------------------
--
# Install OpenSCAP + the SCAP Security Guide profiles, then scan against CIS
L1.
# ------------------------------------------------------------------------
--
sudo apt install  -y  libopenscap8  ssg-debderived
# Debian/Ubuntu content pack
oscap  xccdf  eval  \
       --profile   xccdf_org.ssgproject.content_profile_cis_level1_server
\
       --results   results-fs01.xml
\
       --report    report-fs01.html
\
                   /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml
firefox  report-fs01.html
# Pretty, clickable, and safe to submit
```

Lynis is gentler — a shell script, nothing to sync, nothing to service:

```
sudo lynis audit system  --logfile  /var/log/lynis-fs01.log  \
                         --report-file  /var/log/lynis-fs01.dat
```

Collect the scores, the failed checks, and a **three-column before/after delta table** in your Stage 2 document: *check ID / failed before hardening? / passes after hardening?*.

On Windows, use the DISA STIG Viewer with a Windows 10 or Server 2022 STIG. The hand-auditing will test your patience but teaches you more than automated scanners ever will.

Step 2.7 — Map residual gaps to PIPEDA, Bill C-26, and NIST SP 800-53

NordForge operates in Canada and handles personal information (employee payroll, customer contact details). That triggers:

- **PIPEDA** — the federal privacy law administered by the Office of the Privacy Commissioner of Canada (OPC).
- **Bill C-26** — the *Critical Cyber Systems Protection Act* currently making its way through Parliament; though NordForge may not be in scope unless it supplies critical-

infrastructure clients, the ten cyber-security principles published with the bill are the direction of travel.
- **NIST SP 800-53 Rev. 5** — used as the control library. Controls are organised into families like **AC** (Access Control), **AU** (Audit and Accountability), **IA** (Identification and Authentication), **SC** (System and Communications Protection), **SI** (System and Information Integrity).

In `reference/` you will find a starter mapping template. Each row of your residual-risk register should name at least one control ID (e.g., *AC-6 Least Privilege*, *IA-5 Authenticator Management*, *SI-2 Flaw Remediation*) that the risk relates to. If you want to see how real mappings look, CIS publishes PIPEDA mappings to their Controls for free.

If your team is ambitious, map the *same* residual risks against **PCI-DSS v4.0** (only relevant if NordForge takes card payments — assume they do on the customer portal) and call out which controls are needed for *each* framework. This kind of mapping is exactly what a compliance consultant sells at $1 800 a day.

Step 2.8 — Draft your policies

Three documents, short and sharp (max 5 pages each), stored in `policies/`:

1. **Access Control Policy** — who gets access to what, how is access requested and approved, how is it revoked on exit. Reference NIST **AC-2** (*Account Management*) and **AC-6** (*Least Privilege*).
2. **Acceptable Use Policy** — what users may and may not do on company equipment. Small and human — if your AUP is 24 pages, nobody will read it.
3. **Incident Response Plan — Draft v0.1** — skeleton only; you'll finish it in Stage 4. At minimum: definition of an incident, roles (Incident Commander, Technical Lead, Communicator, Scribe), severity levels, contact list, escalation matrix.

Templates are in `policies/*.md`. Edit them. Don't hand them back verbatim.

5.3 Stage 2 deliverables — what to hand in on 11 Jun

`TeamXX_Stage2.zip` containing:

1. `Stage2_Document.pdf` (est. 20–30 pages):
    - Updated diagram with firewall rule zones coloured in
    - Hardening narrative for Linux, Windows, and AD — with before/after Lynis / SCAP / BloodHound screenshots
    - Vulnerability scan report + CVSS triage spreadsheet
    - Compliance scan reports
    - Framework mapping matrix (PIPEDA / NIST / optional PCI-DSS)
    - Three draft policies
    - Rule-set diff on OPNsense (export *before* and *after*)
    - Short reflection on trade-offs

2. `policies/*.md`
3. `scripts/` — your hardening shell + PowerShell scripts, commented
4. `evidence/openvas_*.csv`, `evidence/scap_*.html`, `evidence/lynis_*.dat`
5. Updated `assets/risk_register.csv` (residual risk column populated)
6. `readme.md`

Grading hotspots: Windows + AD hardening depth, honesty of the framework mapping (don't write "compliant" when you mean "mostly compliant except for this one scary bit"), and demonstrating that your firewall rules actually *do* what your policy says they do — you'll be asked to prove it in the presentation.

---

6. Stage 3 — *Test the defence* (due Thu 18 Jun, Session 12)

Weight: 7 % **Covered courses:** 420-3C2-DW Intrusion Tests (almost entirely) · feedback loop into 420-3C1-DW (vulnerability mgmt) and 420-2C1-DW (cyber defence) **Competency element:** *Validation* of Stage 2 controls. You built it; now attack it and find out where you were wrong.

This is the stage where half of you will find out that you were *not* as hardened as you thought. Welcome to the profession.

6.1 A very important word about authorisation and OPSEC

You will only attack the infrastructure **you built in your own VirtualBox**. Not your partner's home network. Not a lab at college. Not a neighbour's Wi-Fi. Not a cloud VPS. **Your own VMs, on your own laptop, with your own snapshot taken beforehand.** The *Criminal Code of Canada*, section 342.1 — *Unauthorised use of computer* — treats this as a serious offence. Do not give yourself a criminal record to pass a class project.

OPSEC (Operational Security) in this stage is a simulation. In a real engagement you would think about source IP attribution, VPN (Virtual Private Network) chains, and timing. For Stage 3 you show that you *understand* those concepts, even though your traffic all flows over your laptop's loopback. Spin up a Whonix Gateway VM as the gateway for your external Kali if you want the extra point — justify why in one paragraph in the document.

6.2 Rules of Engagement (RoE) — mandatory

Before the first packet leaves Kali, you write and **both partners sign** an RoE document. Template is in `policies/rules_of_engagement.md`. It must include:

- Scope: IPs / subnets / hostnames that are in scope
- Explicit out-of-scope list (everything outside your VLAN10/20/30/40/198.51.100.0/24)

- Allowed techniques and tools (e.g., OpenVAS yes, hping3 rate-limited yes, physical attacks no)
- Prohibited techniques (no destructive payloads, no wiping disks, no real phishing against humans)
- Testing window (dates and times)
- Incident communication — if you break something, you stop, snapshot, message the instructor
- Emergency stop procedure ("Ctrl-C and revert snapshot")
- Signatures and dates

RoE is not bureaucracy for its own sake — on real engagements it's the single document that keeps you out of court when a customer's Exchange server dies at 3 a.m. during your test. Treat it that way.

6.3 Test plan — PTES phases mapped to your timeline

The **PTES (Penetration Testing Execution Standard)** divides a test into seven phases. Map your sessions onto them:

PTES phase	Session	What you do
1. Pre-engagement Interactions	S10 start	RoE, snapshots, lab verification
2. Intelligence Gathering	S10	Passive OSINT (simulated), dig / whois on mock domain
3. Threat Modeling	S10	From Stage 1 threat model, pick 3 most likely attack paths
4. Vulnerability Analysis	S10–S11	OpenVAS scan from **external** Kali, triage
5. Exploitation	S11	Demonstrate 3 successful exploits (see §6.5)
6. Post-Exploitation	S11	Privilege escalation, lateral movement, simulated persistence
7. Reporting	S11–S12	Write the pentest report, include detection evidence

The **CEH (Certified Ethical Hacker)** methodology is slightly different (six phases — Reconnaissance, Scanning, Gaining Access, Maintaining Access, Clearing Tracks, Reporting) but covers the same ground. Pick one framework, say which, and stick to it.

6.4 Reconnaissance (passive + active)

Passive, from outside the perimeter (simulate):

```
# ------------------------------------------------------------------------
--
```

```
# Simulated OSINT on NordForge. The "public" records only exist inside your
lab
# because there is no real NordForge. The point is to exercise the tooling.
# ------------------------------------------------------------------------
--
dig  nordforge.local  ANY   @10.10.30.10                                  # The
mock public DNS
whois  203.0.113.2                                                        #
Yields TEST-NET-3 placeholder
theHarvester  -d  nordforge.local  -b  all                                #
Emails & subdomains (will be empty — that's OK)
amass  enum  -passive  -d  nordforge.local                                #
Documents exercise even if no results
```

Active, from external Kali on 198.51.100.0/24:

```
# Quick TCP SYN sweep of the DMZ, ports 1-65535 on one host (Metasploitable)
nmap  -sS  -p-  --min-rate 2000  --reason  -Pn  \
     -oA  ~/recon/metasploitable-fullport  \
     10.10.30.50

# Version detection + default scripts on the interesting ports only
nmap  -sV  -sC  -p 21,22,23,25,80,139,445,3306,5432,6667,8180  \
     -oA  ~/recon/metasploitable-svc  \
     10.10.30.50

# SMB-focused enumeration -- this will be blocked if your firewall rules from
Stage 2 are right
enum4linux-ng  -A  10.10.30.50

# LDAP anonymous bind check
ldapsearch  -x  -h  10.10.20.10  -b  ""  -s  base  namingContexts
```

Then map every discovered service to an ATT&CK technique. For example: exposed SMB with anonymous binds → **T1135** (*Network Share Discovery*) and **T1021.002** (*SMB/Windows Admin Shares*).

6.5 Exploitation — mandatory demonstrations

You **must** demonstrate, document, and capture evidence of the following three classes of exploit. Choose a specific CVE or technique for each. Screenshots of both the attack and the SIEM alert (or lack thereof — also interesting!) are required.

A. A *network service* vulnerability on Metasploitable 2 — e.g., the vsftpd 2.3.4 backdoor (`CVE-2011-2523`) or the UnrealIRCd backdoor (`CVE-2010-2075`). Both are well-documented in Rapid7's Metasploitable guide. Demonstrate reverse shell, basic system enumeration (`uname -a`, `cat /etc/passwd`), and **stop**. No mining cryptocurrency on the box.

B. A *web application* vulnerability on DVWA or Juice Shop — SQL injection (SQLi), reflected or stored XSS (Cross-Site Scripting), or IDOR (Insecure Direct Object Reference). Use Burp Suite Community Edition to intercept and modify requests; show the before/after of a

successful injection. Tools like `sqlmap` are fine but not a substitute for understanding what's going on underneath — I will ask you.

C. An *Active Directory* vulnerability — at minimum, perform **AS-REP Roasting** or **Kerberoasting** against a user you deliberately left misconfigured before hardening. Show the **hash** you extract (use `Rubeus` or `impacket-GetNPUsers`), then demonstrate that after applying the Stage 2 hardening (set the `DONT_REQ_PREAUTH` flag back to false, and ensure SPNs (Service Principal Names) have AES-only + strong passwords), the same attack fails. That contrast is the point.

Example AS-REP roasting, using Impacket:

```
# ------------------------------------------------------------------------
--
# AS-REP roasting: request Kerberos pre-auth ticket for users that don't need
it
# This returns an AS-REP encrypted with the user's NT-hash-derived key --
crackable.
# ------------------------------------------------------------------------
--
impacket-GetNPUsers  corp.nordforge.local/  \
    -usersfile  ~/recon/users.txt  \
    -dc-ip      10.10.20.10  \
    -request  \
    -format     hashcat  \
    -outputfile ~/loot/asrep_hashes.txt

# Then offline, on your attacker box only:
hashcat  -m 18200  ~/loot/asrep_hashes.txt  /usr/share/wordlists/rockyou.txt
```

And the corresponding Wazuh/Suricata detection. If your sensors did **not** catch the attempt, that is itself a finding — write it down, then tune your rules.

6.6 Post-exploitation — keep it boring

Post-ex is where students get tempted to get creative. Don't. Your job is to document:

- How did you escalate from initial access to higher privilege? Tools like `LinPEAS` and `WinPEAS` are fine. Run, capture output, summarise in plain English. *"I found that `/etc/sudoers` allows `www-data` to run `tar` as root, which lets me break out via GTFOBins and get a root shell."*
- Could you pivot to another VLAN? Yes / no — why? If yes, the segmentation from Stage 2 is wrong and needs fixing.
- Simulated persistence: describe (do not execute) how an attacker could maintain access. For instance, "We would add a scheduled task `cyberSohoLabDemo` which calls back to the C2 (Command and Control) server every 30 minutes." Demonstrate that your EDR / Wazuh catches it.
- Cleanup: **revert to the pre-attack snapshot** at the end of Stage 3.

6.7 IDS evasion — light touch

Try three standard evasion tricks against your own Suricata:

1. Nmap fragmentation (`-f` and `--mtu 16`)
2. Nmap timing slowdown (`-T1` or even `-T0`, which is painful but realistic)
3. `proxychains` over a second SOCKS hop

For each, note: did Suricata fire an alert? Did Wazuh escalate it? If the answer is no, write down what rule you would add. A sample custom rule is in `rules/custom_suricata_rules.rules`.

6.8 The pentest report

In industry this is what the client actually pays for. Six sections, in this order:

1. **Executive Summary** (1 page, no jargon) — what you did, the three most important findings, the headline recommendations. A CFO should be able to read only this page and approve the remediation budget.
2. **Scope, RoE, Methodology** — copy-paste from the RoE document, add PTES phase mapping.
3. **Findings** — one per finding, with:
    - Title, severity (CVSS v3.1 base + environmental)
    - MITRE ATT&CK technique
    - Affected asset
    - Reproduction steps with screenshots
    - Recommended remediation
    - Retest expectation
4. **Attack narrative** — the timeline of the test as a story. Date/time stamped, tied to Wazuh / Suricata alerts where applicable.
5. **Detection gaps** — rules that should have fired but didn't, and what rule to add (this feeds Stage 4).
6. **Appendices** — raw command output, hashes (redacted if they belong to real accounts), full scan files.

A decent template to imitate: the OSCP (Offensive Security Certified Professional) exam report template — it's free to download, even without enrolling.

6.9 Stage 3 deliverables — what to hand in on 18 Jun

`TeamXX_Stage3.zip`:

1. `Stage3_Document.pdf` (est. 25–40 pages) — the pentest report above
2. `roe/rules_of_engagement_signed.pdf`
3. `evidence/` — nmap/openvas outputs, pcaps, screenshots, loot/*.txt (hashes only)
4. `detections/` — Wazuh alert exports and Suricata eve.json snippets

5. `rules/proposed_new_suricata_rules.rules` — rules you would add to close your detection gaps
6. Updated `assets/risk_register.csv` with newly discovered risks
7. `readme.md`

Grading hotspots: the *executive summary* (if it reads like a checklist, you fail the section), the *detection gaps* (what rules would you add, and where are the defensive blind spots?), and whether the AD demonstration convincingly shows **before hardening → attack succeeds → after hardening → attack fails**.

---

7. Stage 4 — *Detect, respond, recover & educate* (due Tue 23 Jun, Session 15)

Weight: 7 % (stage) + 30 % (final document) + 12 % (presentation) = **49 %** of your course grade **Covered courses:** 420-4C1-DW Incident Management · 420-4C2-DW (E4: Raise user awareness) **Competency elements:** EZ08-3 *Manage incidents — detection, resolution, reporting* · EZ08-4 *Raise user awareness*

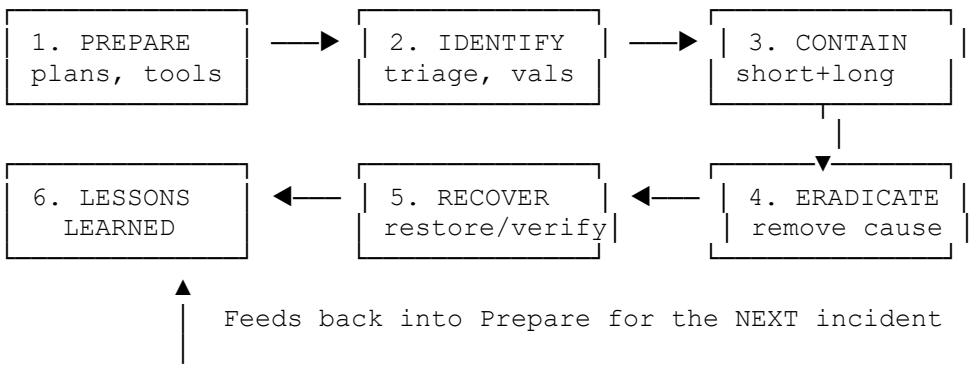
Stage 4 is where the whole project either comes together or falls apart. You will convert the raw Stage 3 attack data into a complete **incident case** that your imaginary SOC worked on end-to-end, and you will produce awareness material that would actually work on real humans.

7.1 Objectives of Stage 4

1. Finalise the **Incident Response Plan (IRP)** — full v1.0 based on NIST SP 800-61 Rev. 2 and the SANS Incident Handler's Handbook.
2. Write **three playbooks**: phishing, ransomware / malware, web application attack.
3. Run a **tabletop + technical IH&R simulation** using the Stage 3 attack as the raw material — go through all six IH&R phases.
4. Collect and preserve **digital evidence** properly (chain of custody).
5. Produce a **post-incident report** in industry format.
6. Produce an **awareness campaign**: a poster, a 2-page user handbook, a 15-minute training session outline, and a phishing-simulation KPI (Key Performance Indicator) plan.
7. Package the **Final Project document** (the comprehensive one, combining Stages 1–4) and the **pre-recorded PowerPoint**.

7.2 The IH&R lifecycle — what you model

NIST SP 800-61r2 defines four phases; SANS expands them to six. You use **six** because it is more forgiving when marking evidence later:



For each phase you document: what you did, who did it, at what time, with what tool, and what evidence you have.

7.3 Build the case from Stage 3 evidence

You already have attack traffic, SIEM alerts, compromised account hashes, exploitation artefacts. Now you put yourself in the *defender's* chair and pretend you didn't know what was coming.

Phase 2 — Identify. Use the Wazuh dashboard and Suricata's `eve.json` to tell a story: at time T, a brute-force alert fires against SSH on FS01. A minute later, an AS-REP roast request is seen against DC01. A minute after that, a Metasploit payload lands on Meta01. Correlate. This is where students either shine or trip — it is easier to run the attack than it is to *reconstruct* it from logs.

Sample Wazuh query to pull all alerts with severity ≥ 10 in a time window:

```
# ------------------------------------------------------------------------
--
# Query Wazuh (Indexer / OpenSearch) for high-severity alerts in the pentest
window
# ------------------------------------------------------------------------
--
curl -k -u  "admin:$WAZUH_PASS"  \
  -H 'Content-Type: application/json'                                    \
  -X POST "https://10.10.40.10:9200/wazuh-alerts-*/_search?pretty"       \
  -d '{
        "size": 500,
        "query": {
          "bool": {
            "filter": [
              { "range": { "rule.level": { "gte": 10 } } },
```

```
              { "range": { "@timestamp": {
                  "gte": "2026-06-17T18:00:00",
                  "lte": "2026-06-17T22:00:00"
                } } }
            ]
          }
        },
        "sort": [{ "@timestamp": "asc" }]
      }'  |  jq '.hits.hits[] | {t: ._source["@timestamp"], r:
._source.rule.description, src: ._source.data.srcip}'
```

Export the results, format them into a **timeline table**, and include that in the post-incident report.

Phase 3 — Contain. Two flavours: *short-term* (block the attacker at the firewall right now, quarantine the VM) and *long-term* (put the compromised host into a dedicated isolation VLAN until forensics is done).

```
# Short-term: block the attacker IP at OPNsense via the API (configd call)
# (The API key pair is stored in /root/.opnsense-api -- never in the repo.)
ATT_IP="198.51.100.50"
configctl  firewall alias add  quarantine  "$ATT_IP"  "stage3-attacker-IP-
quarantine"
configctl  filter reload
```

Long-term containment = move Meta01's NIC to a fresh internal network with no gateway. The VM stays up (for forensics) but cannot talk to anything. Document that move.

Phase 4 — Eradicate. On a real incident you'd rebuild from gold image. In the lab you can still *simulate* eradication: delete the malicious scheduled task / binary / cron job; rotate every credential that touched the compromised host; invalidate Kerberos TGTs by resetting the `krbtgt` account password twice (yes, twice — Microsoft documents why).

Phase 5 — Recover. Restore from the Stage 2 snapshot you *should* have taken (a snapshot, not "I'll rebuild it from memory, I know what was there"). Re-run the vulnerability scan and compliance scan from Stage 2 to prove the host is back to baseline. Turn firewall quarantine off after **all** hardening verified.

Phase 6 — Lessons Learned. Hold a 30-minute blameless post-mortem between yourselves. Write 5–10 lessons. Bullet the ones that translate into concrete changes — new detection rule, new policy, new training module.

7.4 Digital forensics — chain of custody

On a real incident the first minutes are where evidence gets destroyed because people panic and reboot machines. Show that you know better. For the simulation:

- **Memory capture** before shutdown. Use `AVML` on Linux or `DumpIt` on Windows. Hash the output (SHA-256) immediately.

- **Disk image** only after memory. `dd if=/dev/sda of=/media/evidence/fs01.img conv=sync,noerror bs=4M status=progress`. Then hash the image.
- **Log pull** — `/var/log/*`, Windows Event Logs exported as `.evtx`, Wazuh alert exports.
- **Chain of custody form** — who collected what, at what time, with what tool, hash values, where it went after. Template in `policies/chain_of_custody_form.md` (draft in your bundle).

Then you *could* analyse with Volatility 3 on the memory dump, Autopsy on the disk image, and Log2timeline / Plaso on the log archive. You don't need to do a full forensic analysis for this project — but you need to show that you know the tools, you know the order, and you can write one up to a reproducible standard. One screenshot of `volatility3 -f mem.raw windows.pslist` is plenty to demonstrate the workflow.

7.5 Specific incident categories you must touch

The incident management course breaks incidents into categories. For a full mark on Stage 4, your post-incident report must explicitly address at least **four** of the six:

Category	How to fit it into your NordForge case
Malware / ransomware	Simulated ransomware note on FS01 (drop a text file `READ_ME_NOW.txt` by hand during the attack; don't actually encrypt)
Phishing / email	Provide a sample malicious EML (Electronic Mail) file; analyse its headers
Network intrusion (DoS, unauthorised access)	Use your Stage 3 recon + exploitation data
Web application attack	Use your DVWA / Juice Shop exploitation evidence
Cloud	Short paragraph: *"If NordForge's portal were hosted on AWS or Azure, which additional logs would we collect? (CloudTrail, GuardDuty, Azure Activity Log…)"*
Insider threat / endpoint (mobile, IoT)	Short paragraph on how you would detect an employee exfiltrating the customer CSV to a USB stick

The last two can be written-up only (no lab work required) — but they must be thoughtful, not a couple of generic sentences.

Phishing header analysis — worked example

A sample `.eml` is in `awareness/phishing_email_sample.eml`. The headers to read, in order:

```
Return-Path: <bill.dav1s@no-reply-nordforge-pay.com>      ← suspicious domain
lookalike
Received: from ugly-vpshost.biz (203.0.113.77)            ← IP reputation
check on abuseIPDB
DKIM-Signature: ... bh=... b=...                           ← did it pass?
check with `opendkim-testkey`
```

```
SPF: softfail                                                  ← domain owner
says "not me but I won't assert"
From: "Bill Davis — NordForge AP" <accounts.payable@....>  ← display name
spoofing
Subject: URGENT: Please update vendor bank details
```

Tools: `mail-header-analyser`, `MxToolbox header analyzer`. Document what each header tells you and what the attacker did right/wrong.

7.6 The Incident Response Plan — v1.0

Take your Stage 2 draft and finish it. A good IRP for a 200-person company is **15–25 pages**. Structure:

1. Purpose and scope
2. Definitions (incident, event, breach, severity levels SEV-1 to SEV-4)
3. Roles and responsibilities: Incident Commander, Technical Lead, Communications Lead, Scribe, Legal / Privacy Officer contact, External comms (PR + regulators)
4. Detection and triage
5. Communication plan — internal (who tells who, when), external (customers, regulators under PIPEDA breach reporting, the Canadian Centre for Cyber Security (CCCS) via GetCyberSafe, insurance carriers)
6. Containment, eradication, recovery procedures
7. Evidence and chain of custody
8. Lessons learned template
9. Plan maintenance cadence (at minimum annual review, test via tabletop every six months)
10. Appendices — playbooks, contact cards, escalation matrix

Three playbooks, each 3–5 pages, in `policies/playbook_*.md`:

- Phishing — containment decisions, URL detonation (don't click, use urlscan.io or ANY.RUN sandbox), user-reporting workflow
- Ransomware — do-not-pay guidance (CISA joint alert), isolation, backup verification
- Web app attack — WAF (Web Application Firewall) hot-patching, log preservation

7.7 User-awareness campaign

You deliver **four pieces** under `awareness/`:

1. A one-page poster — printable at tabloid size — with five phishing red flags. Draft provided in `awareness/phishing_red_flags_card.html`; make it your own. Colour scheme should be legible, not clever.

2. A two-page user handbook — for the average NordForge employee. Plain language, short sentences, action-oriented. If an accountant cannot read it in 90 seconds and come away with

three things they would do differently tomorrow, rewrite it. Draft in `awareness/user_handbook_2page.md`.

3. A 15-minute training outline — talking points, one realistic phishing example (not stock screenshots — build one from your own sample EML), a live Q&A prompt. Save as `awareness/training_outline_15min.md`.

4. A phishing-simulation KPI plan — if you were going to run a simulated phishing campaign every quarter using GoPhish or a commercial tool like KnowBe4, what would you measure? Suggested KPIs:

- Click-through rate (CTR) — target < 10 % after one year
- Report rate (how many users click *Report Phishing*) — target > 40 %
- Time-to-report — target median < 5 minutes
- Credential-submission rate on landing pages — target < 2 %

Define measurement method, baseline, and cadence.

7.8 Final document — putting it all together

The Final Project document is the combined narrative of Stages 1–4 plus three new things:

- **Executive summary for the CEO** (max 2 pages, no jargon, three headline findings, three headline recommendations, residual risk summary).
- **Return-on-investment argument** — estimate the cost of the controls you implemented (rough order-of-magnitude — license fees, rough engineer-hours, training time) and compare to the expected loss avoidance per the risk register. Ponemon Institute and IBM Cost of a Data Breach publish annual averages you can cite responsibly.
- **Road-map** — what you'd do in the next 90 / 180 / 365 days if this were a real engagement.

Appendix the Stage 1–4 documents. The final document should be **80–150 pages** counting appendices. Any less and you probably skipped a section.

7.9 The pre-recorded PowerPoint presentation

Each partner records **their own half** of the narration. The PowerPoint recording feature is *Slide Show → Record Slide Show → Record from Current Slide* in modern Office / Microsoft 365. Use the built-in timer. Expected total runtime: **20–25 minutes**. Good length per slide: 30–60 seconds.

A working slide deck outline is in `presentation/presentation_outline.md`. Adapt ruthlessly to your team. Each partner narrates roughly half. Make it clear on each slide who is speaking — put a small initial in a corner if needed.

Evaluation criteria for the presentation:

- Clarity — can a non-expert follow along?
- Evidence — do your screenshots and demos actually back your claims?
- Ownership — does each partner convincingly own their half? If I can't tell which of you did the Linux hardening because you both read from the same notes, marks drop.
- Pacing — no dead air, no rambling, no 4-minute slides.

7.10 Stage 4 deliverables — what to hand in on 23 Jun

`TeamXX_Final.zip` — this is your **full and final deliverable**. Structure:

```
TeamXX_Final.zip
├── 00_Executive_Summary.pdf           (2 pages, for the CEO)
├── 01_Final_Project_Document.pdf      (the whole narrative, Stages 1-4
integrated, 80-150 pages)
├── 02_Presentation.pptx               (with narration recorded in-slide)
├── 02b_Presentation_Export.mp4        (PowerPoint export of the narrated
deck as video)
├── 03_Post_Incident_Report.pdf        (the Stage 4 case)
├── 04_Pentest_Report.pdf              (refined from Stage 3)
├── assets/
│   ├── asset_inventory.csv
│   ├── risk_register.csv             (final version)
│   └── cvss_triage.csv                (final version)
├── policies/
│   ├── access_control_policy.md
│   ├── acceptable_use_policy.md
│   ├── incident_response_plan_v1.md
│   ├── rules_of_engagement_signed.pdf
│   ├── playbook_phishing.md
│   ├── playbook_ransomware.md
│   ├── playbook_webapp_attack.md
│   └── chain_of_custody_form.md
├── awareness/
│   ├── poster_phishing.pdf
│   ├── user_handbook_2page.pdf
│   ├── training_outline_15min.md
│   └── simulation_kpi_plan.md
├── scripts/                           (all your custom code, commented)
├── evidence/                          (pcaps, scan outputs, memory dumps,
logs)
├── detections/                        (Suricata + Wazuh rule files, custom
rules)
├── diagrams/                          (final architecture, attack paths,
IH&R lifecycle)
└── readme.md                          (describes every file)
```

Grading hotspots for the Final deliverable:

- **Does the story hold together across all four stages?** The risk register you started in Stage 1 should be the risk register you're closing items against in Stage 4 — with version history visible.

- **Does the presentation *demonstrate* real understanding, not just polished slides?** I can tell within two slides.
- **Executive summary quality.** This is the hardest page in the document. Compress the entire project into two pages that a CEO can act on.

---

8. Evaluation rubric — how the grade is calculated

Percentages for the summative components were given in §1.2. This section details *what* I mark inside each component. Use this document — literally — as a self-assessment checklist before submission.

8.1 Staged submissions (28 %, 7 % each)

Dimension	Weight within stage	What I look for
Completeness of deliverables	25 %	Everything in the Stage's "what to hand in" list is present
Technical correctness	30 %	Tools configured properly, outputs interpreted correctly
Documentation quality	20 %	Clear writing, abbreviations expanded, hyperlinks, screenshots captioned
Integration with prior stages	15 %	Stage 2 builds on Stage 1; no orphan work
Reflection / critical thinking	10 %	Honest about what didn't work; alternative tool justifications

8.2 Final Project document (30 %)

Dimension	Weight	What I look for
Executive Summary	10 %	2 pages, non-technical, actionable
Narrative cohesion across Stages 1–4	20 %	One project, not four bolt-ons
Technical depth and accuracy	25 %	Correct concepts, correct tools, correct interpretation
Evidence quality	20 %	Screenshots, PCAPs, logs, hashes — all referenced from the text
Framework mapping (NIST / PIPEDA / Bill C-26)	10 %	Actual mappings, not name-dropped
Writing, formatting, glossary, hyperlinks	10 %	Literacy standards — up to 10 % deduction per course outline

Dimension	Weight	What I look for
Originality and alternative-tool justifications	5 %	If you substituted, you justified, and you showed code

8.3 Pre-recorded presentation (12 %)

Dimension	Weight	What I look for
Content accuracy and coverage	30 %	Each phase represented; each partner owns their half
Clarity of explanation	25 %	A non-expert should be able to follow
Demonstrations / evidence	20 %	Live-looking screenshots, not stock art
Delivery quality	15 %	Audible, paced, no dead air, no monotone
Visual design	10 %	Legible slides, consistent branding, minimal walls of text

8.4 Final written exam (30 %)

A three-hour in-class exam on **26 June**, covering material from *all seven* integrated courses plus the capstone. Expect:

- 30 MCQ (Multiple-Choice Questions) — 30 marks
- 6 short-answer (2–3 sentences each) — 30 marks
- 2 scenario-based questions — 40 marks (one defensive, one incident response)

Open notes? No. Open mind? Mandatory.

---

9. Alternative tools policy — when you want to swap mine out

You are adults. You are also about to be paid professionals. Part of being one is *choosing* your tools, not being handed them. So: you are entirely free to swap any recommended tool or platform for another, under three conditions.

1. **Document the swap.** One paragraph, in the Stage document where it first appears, naming both my recommendation and your substitute, and explaining the engineering reason for the choice. Good reason: *"We used Proxmox VE instead of VirtualBox because we wanted to test nested-virtualisation scenarios and Proxmox gives us cleaner snapshots on LVM-thin storage."* Bad reason: *"We like it better."*
2. **Show the code if you wrote it.** If your substitute is something *you* built — a custom SIEM parser, a PowerShell script to replace `SharpHound`, anything — the source code

goes in `scripts/` with thorough comments and a `README` explaining how to run it. I will read the code. I will run it. If the code does not do what the document says it does, marks drop.

3. **Prove "more complex" is not just "different".** If you rebuild the infrastructure with more moving parts — e.g., you add a separate Postgres server, Redis cache, HAProxy front-end, and Keepalived floating IP in front of the web tier — you must include a one-page architectural justification explaining *why* that complexity buys you a better security lesson and not just more breakage. Real example of a good justification: *"We split the DMZ into a front-end tier (Nginx reverse proxy) and a back-end tier (application + DB), with inter-tier traffic restricted to specific ports, so we can demonstrate a tiered compromise and show how far an attacker gets through each tier."*

Common substitutions I accept without a grimace, if justified:

My recommendation	Reasonable alternative	Why you might prefer it
OPNsense	pfSense CE	Longer historical mindshare; more YouTube content
OPNsense	IPFire	Cleaner zone model for beginners (Red/Green/Blue/Orange)
Wazuh	Security Onion	Richer network-centric analytics
OpenVAS / Greenbone	Nessus Essentials	Better UI, industry-standard reports — limited to 16 hosts
DVWA + Juice Shop	WebGoat + Mutillidae	Slightly different vulnerability set
Kali	Parrot OS Security	Lighter footprint, some students prefer its defaults
VirtualBox	VMware Workstation Pro (free for personal since May 2024)	Slightly smoother on Windows hosts
PowerPoint	LibreOffice Impress	Free; can export narrated video

What I will **not** accept: tools that are closed-source, paid-only, and inaccessible to your classmates (so they can't reproduce your work); tools whose licence forbids educational use; anything you can't explain in the Q&A.

---

10. Downloadable artefacts that ship with this brief

The ZIP you receive alongside this Markdown contains starter files and reference material. Each is a **starting point**, not a finished deliverable. Fill, modify, and cite.

Path	What it is
bash/01_lab_sanity_check.sh	Pre-flight check for your VM fleet — interfaces up, DNS reachable, time sync
bash/02_linux_hardening.sh	Opinionated Linux hardening baseline script
bash/03_compliance_spot_checks.sh	Manual compliance checks (SSH config, file perms, users)
bash/04_collect_triage_artifacts.sh	IR artefact collector — logs, processes, netstat, hashes
powershell/01_windows_baseline.ps1	Windows baseline: SMBv1 off, LM auth disabled, audit policy
powershell/02_ad_audit.ps1	AD hygiene audit: kerberoastable SPNs, AS-REP, stale accounts
powershell/03_ir_triage.ps1	Windows IR triage: volatile data, event logs, autoruns
policies/access_control_policy.md	Starter Access Control policy
policies/acceptable_use_policy.md	Starter Acceptable Use policy
policies/incident_response_plan.md	Starter IRP template
policies/rules_of_engagement.md	RoE template for Stage 3
spreadsheets/asset_inventory_template.csv	Asset inventory CSV
spreadsheets/risk_register_template.csv	Risk register CSV
spreadsheets/cvss_triage.csv	CVSS triage CSV
spreadsheets/stakeholder_communication_matrix.csv	Who talks to whom during an incident
spreadsheets/evaluation_rubric.csv	Machine-readable version of the grading rubric
reference/abbreviations_glossary.csv	Long-form glossary of every abbreviation you will use
reference/mitre_attack_mapping.csv	Sample ATT&CK technique table
reference/log_sources_and_parsers.csv	Which logs matter and which parser reads them
diagrams/network_topology.txt	ASCII reference topology
diagrams/attack_path.txt	ASCII attack path example
diagrams/ih_r_lifecycle.txt	ASCII IH&R lifecycle
rules/custom_suricata_rules.rules	Starter custom Suricata rules
rules/sample_yara.yar	Example YARA rule
awareness/user_handbook_2page.md	User handbook starter
awareness/phishing_red_flags_card.html	Printable red-flag card

Path	What it is
`awareness/phishing_email_sample.eml`	Sample phishing EML for header analysis
`presentation/presentation_outline.md`	Slide-by-slide presentation outline

---

11. Glossary of abbreviations

A more exhaustive glossary is in `reference/abbreviations_glossary.csv`. A short, representative list for quick reference:

- **AD** — Active Directory (Microsoft directory service)
- **ACL** — Access Control List
- **API** — Application Programming Interface
- **APT** — Advanced Persistent Threat
- **AS-REP** — Authentication Service Response (Kerberos)
- **ATT&CK** — Adversarial Tactics, Techniques, and Common Knowledge
- **AUP** — Acceptable Use Policy
- **BYOD** — Bring Your Own Device
- **C2** — Command and Control (attacker infrastructure)
- **CCCS** — Canadian Centre for Cyber Security
- **CEH** — Certified Ethical Hacker
- **CIA Triad** — Confidentiality, Integrity, Availability
- **CIS** — Center for Internet Security
- **CISA** — Cybersecurity and Infrastructure Security Agency (USA)
- **CVE** — Common Vulnerabilities and Exposures
- **CVSS** — Common Vulnerability Scoring System
- **DMZ** — Demilitarized Zone (network segment)
- **DNS** — Domain Name System
- **DoS / DDoS** — (Distributed) Denial of Service
- **DKIM** — DomainKeys Identified Mail
- **DVWA** — Damn Vulnerable Web Application
- **EDR** — Endpoint Detection and Response
- **ELK** — Elasticsearch, Logstash, Kibana (stack)
- **EML** — Electronic Mail (file format for email messages)
- **FAIR** — Factor Analysis of Information Risk
- **FIM** — File Integrity Monitoring
- **GDPR** — General Data Protection Regulation (EU)
- **GPO** — Group Policy Object
- **HIDS** — Host-based Intrusion Detection System
- **HIPAA** — Health Insurance Portability and Accountability Act (USA)
- **IDS / IPS** — Intrusion Detection / Prevention System
- **IH&R** — Incident Handling & Response
- **IOC** — Indicator Of Compromise
- **IRP** — Incident Response Plan

- **ISO** — International Organization for Standardization
- **KEV** — Known Exploited Vulnerabilities (CISA catalog)
- **LAPS** — Local Administrator Password Solution
- **LDAP** — Lightweight Directory Access Protocol
- **LPE** — Local Privilege Escalation
- **MFA** — Multi-Factor Authentication
- **MITM** — Man-In-The-Middle
- **MITRE** — MIT Research & Engineering (nonprofit)
- **NAT** — Network Address Translation
- **NIDS** — Network Intrusion Detection System
- **NIST** — National Institute of Standards and Technology (USA)
- **NVD** — National Vulnerability Database
- **OPC** — Office of the Privacy Commissioner of Canada
- **OPSEC** — Operational Security
- **OS** — Operating System
- **OSINT** — Open-Source Intelligence
- **OWASP** — Open Worldwide Application Security Project
- **PCAP** — Packet Capture file
- **PCI-DSS** — Payment Card Industry Data Security Standard
- **PIA** — Privacy Impact Assessment
- **PIPEDA** — Personal Information Protection and Electronic Documents Act (Canada)
- **PoC** — Proof of Concept
- **PTES** — Penetration Testing Execution Standard
- **RBAC** — Role-Based Access Control
- **RoE** — Rules of Engagement
- **RCE** — Remote Code Execution
- **SCAP** — Security Content Automation Protocol
- **SIEM** — Security Information and Event Management
- **SLA** — Service Level Agreement
- **SMB** — Server Message Block
- **SOC** — Security Operations Center
- **SPF** — Sender Policy Framework
- **SPN** — Service Principal Name
- **SQLi** — SQL Injection
- **SSH** — Secure Shell
- **STIG** — Security Technical Implementation Guide
- **STRIDE** — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
- **TLS / SSL** — Transport Layer Security / Secure Sockets Layer
- **UFW** — Uncomplicated Firewall
- **VLAN** — Virtual Local Area Network
- **VM** — Virtual Machine
- **VPN** — Virtual Private Network
- **WAF** — Web Application Firewall
- **XSS** — Cross-Site Scripting
- **YARA** — Yet Another Recursive Acronym (pattern-matching tool)

12. Hyperlinks reference section

All the clickable references used in this brief, in one place, with a short note explaining why each one matters. When you cite them in your own Stage documents, do the same — a raw URL on its own is worth less than a URL with a sentence of context.

12.1 Standards, frameworks, and regulators

- **NIST SP 800-30 Rev. 1** — *Guide for Conducting Risk Assessments.* The foundation document for the 5×5 risk matrix and likelihood/impact language.
- **NIST SP 800-53 Rev. 5** — *Security and Privacy Controls for Information Systems and Organizations.* The control library you will map your residual risks to.
- **NIST SP 800-61 Rev. 2** — *Computer Security Incident Handling Guide.* The four-phase IH&R lifecycle source.
- **ISO 31000:2018** — International risk management standard; provides the vocabulary (risk, residual risk, treatment).
- **ISO 27005:2022** — *Information security risk management*, aligned with ISO 27001.
- **SANS Incident Handler's Handbook** — the six-phase IH&R version used in this brief.
- **PTES** — Penetration Testing Execution Standard; methodology for Stage 3.
- **MITRE ATT&CK** — adversary tactics and techniques knowledge base.
- **MITRE D3FEND** — defensive counterpart to ATT&CK.
- **MITRE CVE** — Common Vulnerabilities and Exposures list.
- **NVD** — U.S. National Vulnerability Database; CVE enrichments + CVSS scores.
- **FIRST CVSS v3.1 calculator** — scoring tool.
- **CISA KEV catalog** — vulnerabilities confirmed exploited in the wild.
- **CISA** — U.S. Cybersecurity and Infrastructure Security Agency.
- **CCCS (Canadian Centre for Cyber Security)** — Canadian federal cyber agency.
- **Get Cyber Safe (Canada)** — CCCS-run public awareness portal; useful for your awareness campaign.
- **PIPEDA** — Canadian federal privacy legislation.
- **OPC (Office of the Privacy Commissioner of Canada)** — the regulator.
- **PIPEDA breach response guide** — what to do after a breach under PIPEDA.
- **Bill C-26** — Canadian *Critical Cyber Systems Protection Act* (pending).
- **PCI-DSS v4.0** — payment-card data security standard.
- **PCI Security Standards Council** — PCI-DSS publisher.
- **OWASP** — Open Worldwide Application Security Project home.
- **OWASP Top 10** — canonical web-app risk list.
- **OWASP Threat Dragon** — free threat modelling tool.
- **CIS Benchmarks** — practical hardening guides per OS.
- **DISA STIGs** — Defense Information Systems Agency security guides.
- **Criminal Code of Canada, s. 342.1** — unauthorised use of computer.

12.2 Tools and platforms

- **Oracle VirtualBox** — our reference hypervisor.
- **VMware Workstation Pro** — free for personal use; accepted alternative.
- **OPNsense** — primary firewall.
- **pfSense CE** — accepted alternative firewall.
- **IPFire** — zone-based firewall alternative.
- **Kali Linux** — offensive distro.
- **Parrot OS Security** — alternative offensive distro.
- **Whonix** — anonymity gateway.
- **Ubuntu** — primary Linux server distro.
- **FreeBSD** — OPNsense's underlying OS.
- **Wazuh** — HIDS/SIEM.
- **Security Onion** — alternative SIEM / NSM.
- **Suricata** — NIDS / NIPS engine.
- **Zeek** — network security monitor / metadata parser.
- **Emerging Threats Open ruleset** — free Suricata rules.
- **Metasploit** — exploitation framework.
- **Metasploitable 2** — vulnerable target VM.
- **DVWA** — vulnerable web app.
- **OWASP Juice Shop** — vulnerable web app.
- **OWASP WebGoat** — vulnerable web app.
- **Nmap** — network scanner.
- **Wireshark** — packet analyser.
- **Burp Suite** — web proxy / pentest tool.
- **OpenVAS / Greenbone CE** — vulnerability scanner.
- **Tenable Nessus Essentials** — commercial VS, free tier for up to 16 hosts.
- **OpenSCAP** — compliance scanning.
- **Lynis** — Unix system audit.
- **Fail2Ban** — brute-force blocker.
- **UFW** — Uncomplicated Firewall.
- **BloodHound CE** — AD attack path graphing.
- **Impacket** — Python SMB/Kerberos tools.
- **hashcat** — password cracker.
- **Hydra** — brute-force login tool.
- **GTFOBins** — Unix binaries useful in escape / privilege escalation.
- **PEASS-ng (LinPEAS / WinPEAS)** — privilege escalation scanners.
- **Volatility 3** — memory forensics.
- **Autopsy** — disk forensics.
- **Plaso / Log2timeline** — log timelining.
- **AVML (Azure Virtual Memory Linux)** — Linux memory dumper.
- **urlscan.io** — URL sandbox.
- **ANY.RUN** — interactive malware sandbox.
- **MxToolbox email headers** — online email header analyser.

- **Microsoft Header Analyzer (MHA)** — Microsoft's header tool.
- **GoPhish** — open-source phishing simulation.
- **KnowBe4** — commercial security-awareness vendor.
- **HashiCorp Vault** — secrets management.
- **KeePassXC** — local password manager.
- **draw.io / diagrams.net** — free diagram tool.
- **Excalidraw** — sketch-style diagram tool.
- **Rapid7 Metasploitable exploitability guide** — reference for Stage 3 exploits.
- **Ponemon Institute** — IT security research publisher.
- **IBM Cost of a Data Breach Report** — annual breach-cost figures.

12.3 Dawson College and ISEP references

- **Dawson College** — the college home page.
- **Dawson ISEP (Institutional Student Evaluation Policy)** — the evaluation policy you signed at enrolment.
- **Dawson Code of Conduct** — the conduct policy.
- **BAnQ — Bibliothèque et Archives nationales du Québec** — free access to O'Reilly, LinkedIn Learning, etc.

12.4 YouTube channels worth your time

Claude is not quoting from these — they are simply high-signal creators I recommend. Always verify what you hear on YouTube against the official documentation of any tool you use.

- **Lawrence Systems** — pfSense / OPNsense / firewall content.
- **Network Chuck** — beginner-friendly networking and security.
- **John Hammond** — malware analysis and CTF walk-throughs.
- **IppSec** — HackTheBox machine walk-throughs; the best pentest narration on the platform.
- **13Cubed** — digital forensics and incident response.

Usage note: citing a YouTube video in your document is fine under fair dealing of the Canadian *Copyright Act* for purposes of research and education. Link to the video, note the creator and the timestamp you are referencing, and do not re-upload their content.

12.5 Verification — check what you click

You will download a lot of tools in this project. Before you run anything, run the installer through **VirusTotal — URL scanner** and then **VirusTotal — file scanner**. If any AV engine flags the file as anything other than a riskware classification, **stop and ask**. Better paranoid for five minutes than compromised for five months.

---

13. Disclaimer — read carefully

Intellectual Property. This document, its illustrations, scripts, spreadsheets, reference tables, policies, awareness material, and all derivative works produced from it are the intellectual property of **Cyber.SoHo Educational Hub** ("Cyber.SoHo"). They are provided to enrolled students of the *Cybersecurity: Prevention and Intervention — LEA.D8* program for educational purposes only. Redistribution, republication, commercial use, or inclusion in any other training material without the written consent of Cyber.SoHo is prohibited.

No affiliation. Cyber.SoHo is not affiliated with, endorsed by, or sponsored by any vendor, organisation, standards body, regulator, open-source project, or online service named or linked in this brief. All trademarks, product names, and registered marks belong to their respective owners. References to commercial products are made for educational and illustrative purposes only and do not constitute an endorsement. This document does not imply any partnership with Dawson College beyond the specific contractual teaching engagement of the named instructor, nor with Schiller International University beyond the affiliation of Cyber.SoHo's programmes.

Copyright. The organisations, standards documents, laws, and third-party tools referenced in this document are the copyright of their respective owners. Where linked, the link points to the official site or document of record. No portion of any copyrighted source is reproduced verbatim in this document beyond what is permitted by fair dealing provisions of the Canadian *Copyright Act* for the purposes of research and private study. Students are responsible for respecting the licence terms of every tool they download, every rule set they import, and every dataset they use during the project.

External links and software. Hyperlinks in this document point to third-party websites that Cyber.SoHo does not control and cannot warrant. Content, availability, certificates, and safety of linked sites may change without notice. **Before downloading or executing any tool, package, script, or binary referenced in this project**, students are required to verify the file and its source using — at minimum — **VirusTotal (URL)** and **VirusTotal (file upload)**. If any VirusTotal engine flags the file as malicious (excluding generic riskware / PUA classifications on known dual-use tools such as Nmap or Mimikatz), students must not run it and should contact the instructor.

Liability, ethics, and lab safety. This is a cybersecurity course. The techniques, tools, code, and methodologies described here — penetration testing, privilege escalation, password cracking, network sniffing, web application exploitation, evasion, phishing simulation, and others — are **powerful and, outside of the authorised lab, unlawful**. Students agree and acknowledge:

- All exercises are to be performed **exclusively** inside their own isolated VirtualBox (or equivalent) lab environment, on virtual machines that they own, against targets that they have explicitly authorised themselves to test via the signed Rules of Engagement.
- Performing any of these techniques against systems, networks, accounts, or data outside the authorised lab — including but not limited to: other students' machines, the college's network, any employer's systems, friends' or family members' devices, cloud accounts, or

any third-party service — **is a criminal offence** in Canada under s. 342.1 of the Criminal Code, and in most other jurisdictions under comparable law.
- Cyber.SoHo, the named instructor, Dawson College, and Schiller International University **disclaim any and all liability** for damages, legal consequences, data loss, reputational harm, financial loss, or any other consequence arising from the use or misuse of the material in this document outside the authorised lab environment. Students take full personal responsibility for their actions.
- Students are further expected to conduct themselves in accordance with the professional-ethics expectations of the cybersecurity industry — including but not limited to the (ISC)² Code of Ethics, the EC-Council Code of Ethics, and the SANS Code of Ethics.
- By submitting Stage 1 of this project, students confirm they have read, understood, and accept the terms of this disclaimer.

Respect the tools. Respect the profession. Respect each other. Build something real.

Cyber.SoHo Educational Hub — From the Industry to the Classroom — Building the IT's Future, Today and Together!

End of brief. Go build.